

Milestone 3 — Add the Eyes

The same `GridWorld`, seen only as pixels, through a frozen pretrained ViT.

Status: ✓ COMPLETE — “Done when” satisfied

Sprint: M3

Date: 2026-07-27

Author: Phillip

Branch: `m3-implementation`

M3 swapped the **sense organ and nothing else**. `GridWorldEnv` learned to draw itself as a 224×224 picture, a wrapper replaced the `(row, col)` observation with that picture, and a frozen pretrained ViT-Tiny turned it into 192 numbers for PPO to read. The first full run scored `+0.905` and looked like a clean success — it wasn’t. The agent was **completely blind** and had never used the picture at all; it had memorised a single coin flip that happened to solve the maze. Finding that took three diagnostics, and fixing it meant redesigning the **Ground**, not the eyes. After the fix a genuinely sighted agent reached **+0.99 with a 100% goal rate** and the guardrail printed PASS. Every change was made with wrappers, so `GridWorldEnv` was never edited beyond learning to draw — and M2’s tests stayed green the whole way.

6 / 6

BRICKS DONE

41

TESTS PASSING
(+1 SKIPPED)

+0.99

TRAINED MEAN
REWARD

100%

GOAL RATE (GREEDY)

19.2 min

CPU TRAINING, 20K
STEPS

0

CONTRACT CHANGES

1 Accomplishments — the 6 bricks

#	BRICK	FILE(S)	DONE WHEN...	STATUS
0	The guardrail — written first The finish line, committed before any pixel code existed. Measures the random baseline live every run .	<code>scripts/train_gridworld_vit.py</code> (its verdict)	Beat the live baseline by +0.40, reach the goal in ≥80% of greedy episodes, shapes unchanged.	DONE
1	GridWorld draws itself A blocky 5×5 board, upscaled on purpose. The old text renderer still works and stays the default.	<code>gridworld.py</code> → <code>_render_rgb()</code> <code>tests/test_gridworld_pixels.py</code>	<code>render_mode="rgb_array"</code> returns a <code>(224,224,3)</code> uint8 image; moving changes it.	DONE
2	The pixel wrapper (VisionPerception) The env never learns the wrapper exists. The agent can no longer see its coordinates at all — that’s the point.	<code>perception.py</code> → <code>PixelObservation</code> <code>tests/test_pixel_observation.py</code>	obs is the image; <code>reset()</code> →2-tuple, <code>step()</code> →5-tuple, <code>check_env</code> clean.	DONE
3	The borrowed eyes fit Pretrained <code>vit_tiny_patch16_224</code> , backbone frozen — 0 trainable parameters . Borrowed eyes stay borrowed.	<code>vit_extractor.py</code> → <code>ViTTinyFeaturesExtractor</code> <code>tests/test_vit_extractor.py</code>	One picture in, 192 numbers out; the width PPO reads matches the width produced.	DONE
4	Train on pixels — the experiment Caught the blind agent, then fixed the Ground with wrappers only: random start, centre goal, 25-move budget.	<code>scripts/train_gridworld_vit.py</code> <code>tests/test_random_start.py</code>	The script prints PASS — this is the milestone “Done when” .	DONE
5	Wire into TUI + changelog New item <code>[5] Train GridWorld with ViT eyes</code> . The old <code>[5]</code> lost its misleading “M3+” label — two items claiming M3 is how you launch the wrong one.	<code>tui.py</code> · <code>setup.py</code> <code>docs/CHANGELOG.md</code>	The menu item launches the script; all 8 script paths resolve.	DONE

2 The “Done when” — the finish line, written first

What a “Done when” is, and why it goes first

A “**Done when**” is one sentence, written *before* any code, that says exactly what would count as finished. It goes first for a single reason: **so it can’t be quietly softened later**, when the results come in awkward and a smaller bar starts to look reasonable.

M3’s, verbatim from `M3_ToDo.md` :

*“PPO, seeing **only pixels** through the pretrained ViT, beats the random baseline and reaches the goal consistently — and `reset()` / `step()` still return the exact same shapes they did in M2.”*

Brick 0 turned that sentence into three **testable** conditions, all of which had to hold at once:

- Mean reward beats the random baseline **measured live in the same run**, by $\geq +0.40$.
- The agent reaches the goal in $\geq 80\%$ of greedy evaluation episodes.
- `reset()` / `step()` return M2’s shapes, unchanged.

Verdict vs. the definition of done

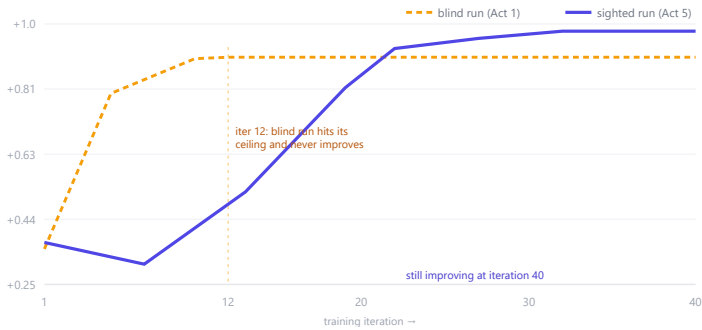
Live random baseline	+0.48
Required margin	$\geq +0.88$
Trained mean reward	+0.99
Goal reached (greedy)	100% (needs $\geq 80\%$)
Contract shapes	unchanged
Mean steps to finish	2.2 (optimum ~ 2.5)

✅ **PASS.** All three conditions held at once — **M3’s “Done when” is satisfied**. Note the bar was unusually **harsh**: a noisy 50-episode baseline drew +0.48 against a true $\sim +0.36$, putting the target at +0.88 against a ceiling of $\sim +0.985$ — about 0.10 of headroom. It was **deliberately not adjusted mid-milestone**.

The guardrail earned its keep twice. Once by **catching the blind agent** — the training log read `+0.905` and rising while the verdict printed **FAIL**, and the verdict was right. And once by **refusing to be moved** when the bar turned out harsher than intended.

3 The result — two runs, one lesson

Training reward per iteration — blind run vs. sighted run



Source: `run-logs/m3_run_20k_FAIL.log` and `run-logs/m3_run_vision_PASS.log` (`ep_reward_mean`, 20,000 timesteps each, frozen ViT-Tiny, CPU). **Compare the shapes.** The blind run rockets to its ceiling by iteration 12 and flatlines for 28 more — it had found everything there was to find, and it wasn't much. The sighted run **dips first** (0.371 → 0.307, as random exploration stops paying off), then climbs steadily and is **still improving at the end**. That is what learning to see looks like: slower to start, and it keeps going.

The two runs, side by side

ACT 1 → ACT 5

Verdict	FAIL → PASS
Trained reward	-1.00 → +0.99
Goal rate (greedy)	0% → 100%
Mean steps	100.0 → 2.2
Episode length	10.5 → 3.10
Wall clock	19.4 → 19.2 min

Only the Ground changed between these two runs — same 20,000 timesteps, same frozen ViT-Tiny, same PPO settings, same code for the eyes.

Act 1's **exactly** `-1.00` reward and **exactly** `100.0` steps across all 20 episodes is what gave the bug away. Numbers that clean are never a weak agent — a bad-but-real policy varies.

4 The bug that defined the sprint — an agent with its eyes shut

Diagnosis — the policy ignored its input entirely

Walking the trained agent through the grid and printing its action probabilities gave the **identical** distribution at every single square:

```
step  pos  action  probs [UP DOWN LEFT RIGHT]
0  (0,0)  DOWN   [0.001 0.533 0.  0.466]
2  (2,0)  DOWN   [0.001 0.533 0.  0.466]
4  (4,0)  DOWN   [0.001 0.533 0.  0.466]
5  (4,0)  DOWN   [...] <- stuck on the wall
```

A fixed **53% DOWN / 47% RIGHT** coin flip, played regardless of what it saw. Tracing where the signal died:

STAGE	VARIATION ACROSS CELLS
Raw pixels	0.34369
ViT features (192-dim)	0.03871
Policy logits (4)	0.00000

The pictures were plainly different, the ViT compressed that difference to ~4% (cosine similarity **0.998–0.999**), and the policy head discarded what was left. A hand-written **blind** agent playing that same coin flip scored **+0.907** — matching the “trained” agent exactly.

Root cause — the Ground, not the eyes

The original maze started the agent at (0,0) with the goal in the **corner** (4,4). “Always move down or right” walks you into that corner from *any* square, because bumping a wall merely stops you. **Vision was never required to win**, so PPO took the cheaper path. Comparing a blind agent against a sighted oracle across four task designs settled it:

TASK VARIANT	BLIND	SIGHTED	GAP
A fixed start, corner goal (original)	+0.906	+0.930	+0.024
B random start, corner goal	+0.947	+0.967	+0.019
C random start, centre goal ✓	-0.748	+0.985	+1.733
D random start, random goal	-0.665	+0.976	+1.641

The most surprising result of the milestone is row B. Randomising the start — the obvious fix, and the one tried first — made things worse. **The corner is the trap, not the fixed start.** Variant C was chosen as the smaller step: the agent must learn to see *itself*, and the goal stays put.

A second, independent problem then surfaced: with 100 moves on a 25-square grid, a **random** walk finds a centre goal 94% of the time. Tightening the budget to **25 moves** made the referee discriminate — random and blind agents both fail it, while a sighted agent needs at most 4 moves.

Both changes were necessary; neither was sufficient. With the goal back in the corner and a tight 15-move budget, a blind agent still scored **99%**. Everything was fixed with **wrappers** (`RandomStart`, `make_vision_task()`) — `GridWorldEnv` was never edited.

5 Retrospective — what went well & what didn't

● What went well

- **The guardrail did exactly its job.** Brick 0 was written before any pixel code existed, and it caught a failure the reward curve had disguised as a success. The training log said `+0.905` and rising; the verdict said **FAIL**. The verdict was right.
- **The contract never moved.** Pixels, random start, centre goal, step cap — all of it done with **wrappers**. `GridWorldEnv` gained one drawing method and was otherwise untouched, so M2's tests stayed green for the entire milestone. Zero changes to PPO.
- **Greedy evaluation exposed what sampling hid.** The blind policy looked fine when actions were sampled and fell apart under `argmax`. A stochastic-only verdict would have printed a triumphant PASS on a blind agent.
- **Suspiciously clean numbers were treated as a clue.** Exactly `-1.00` and exactly `100.0` read as a systematic cause, not a weak agent. Chasing that instead of "just train longer" is what found the bug.
- **The fix was measured, not guessed.** Four task designs and six step-budgets were compared against blind and sighted agents *before* anything was changed — and the results overturned the intuitive fix.
- **The bug is now a one-second test.** `tests/test_random_start.py` pins "this Ground cannot be solved blind." What cost a 20-minute run is caught instantly if anyone moves the goal back to a corner.

● What didn't go well / what to do better

- **The first fix was the wrong one.** Randomising the start was the obvious move and it changed nothing — a blind agent still scored `+0.947` at 100%. Time went into the intuitive fix *before* measuring which variant actually needed eyes. **Better:** measure first, then fix.
- **A 20-minute run bought a worthless number.** The entire first training run proved nothing, because nobody checked whether the task *required* vision before training on it. That check takes one second. **Better:** blind-vs-sighted gap becomes a pre-flight check on every new Ground.
- **The baseline sample is noisy.** `BASELINE_EPISODES = 50` drew `+0.48` against a true `~+0.36`, setting the bar 0.12 harsher than intended and leaving only `~0.10` of headroom. Deliberately **not** changed mid-milestone — moving the bar is what Brick 0 exists to prevent. **This is an M4 fix.**
- **The borrowed eyes are the weak link.** Frozen ImageNet ViT features stay `~0.998` cosine-similar across agent positions — it compressed a 34% pixel difference down to 4%. PPO learned to read them once it had a reason to, but the features were never the strong part of this pipeline. The documented next rung is unfreezing the last ViT block.
- **Housekeeping surfaced two bugs hidden for months.** The full suite had *never run* — `test_input.py` blocked collection on an `input()` call, which meant `test_logger.py` had never executed and was concealing a leaked file handle and a shared-logger bug. Separately, `rich` was imported by `tui.py` but listed in **no** dependency file, so `python main.py` had been dying into its error fallback.
- **The evidence isn't fully reproducible.** The two diagnostic scripts analysed the *blind* model, which the successful run overwrote. Outputs are preserved verbatim in `run-logs/`, but they cannot be regenerated. **Better:** archive the model, not just the console output.
- **Still no CI.** Carried over unresolved from the M2 retrospective: tests pass locally but nothing enforces them on push, and this work again sits on an unmerged branch.



The one idea to take away: a rising reward curve proves the agent found *some* winning strategy. **It says nothing about whether that strategy used the eyes.** The only thing that can prove perception matters is the **gap between a blind agent and a sighted one on that exact task** — and that gap costs one second to measure, against twenty minutes to train a number you can't trust.

6 Housekeeping — the audit between Brick 4 and Brick 5

A non-brick session that changed **no milestone behaviour**. All 62 tracked files were audited for dead weight, and fixing the collection block exposed a logger that had never been tested.

Repo audit — 4 files

FILE	ACTION	WHY
tests/test_input.py	→ scripts/	Not a test: interactive, needs Stardew open. This is why the full suite could never run.
src/.../events.py	deleted	11 lines, no code, zero importers.
src/.../dependencies.py	deleted	93 lines, zero importers.
CMakeLists.txt	deleted	Zero references; build section commented out and naming files that don't exist.

Kept on purpose: Track B (`env_vit.py`, `screen.py`, `interface.py`, ~1,900 lines) — `ONBOARDING.md` §5 retains it as M3/M5 reference — and `docs/m3/experiments/`, already archived with its own README.

Logger — two bugs, both previously invisible

- **Leaked file handle.** `FileHandler` was opened and never closed. Silent on Linux; on Windows it surfaces far away as a `PermissionError` when something later deletes the file.
- **Shared logger.** `getLogger("GameTrainer")` returns one process-wide object, and the `if not handlers` guard meant a *second* `Logger` added no handler — so it wrote into the **first** one's file and silently ignored the `log_dir` passed to it.
- Fix: handler kept per-instance, `close()` detaches it, stale handlers cleared on construction, plus `__enter__` / `__exit__`. The public API is unchanged, so no caller needed editing. Both new tests were **verified red** against the old logger.

Result: `pytest tests/` runs unaided for the first time — **41 passed, 1 skipped** (the M2 end-to-end, skipped by design as an experiment). `ruff check .` clean. Test count went 1 → 4 for the logger alone.

7 Scope discipline — what M3 deliberately did *not* build

✗ No screen capture

We draw `GridWorld` ourselves. No real game window, no `mss` capture, no real key presses — that's **M5**.

✗ No reward abstraction

Reward logic still lives inside `GridWorldEnv`. No `Profile` / `RewardCalculator` — that's **M4**.

✗ No fine-tuning the ViT

The eyes are **borrowed and frozen** — 0 trainable parameters in the backbone. Only the policy head learned.

✗ No edits to PPO or the env

"If we have to touch PPO to make this work, we did it wrong." We didn't. Every change was a wrapper.

8 Next steps

Immediate (close out M3)

- **Commit** `docs/m3/M3_Cleanup.md` — currently the only untracked file in the tree.
- **Merge** `m3-implementation` → `main` via PR (**14 commits ahead**), mirroring the M1/M2 flow.
- **Add CI to run** `pytest` **on push** — carried over from the M2 retrospective and still open. Now that the suite actually runs unaided, this is finally unblocked.
- Optional: archive the trained model alongside each run log, so diagnostics stay reproducible.

Next milestone → M4: "Swap the Reward"

- **Raise** `BASELINE_EPISODES` **above 50** — the known-noisy measurement, deliberately deferred out of M3 rather than changed mid-milestone.
- Move reward logic **out** of the env into `Profile` / `RewardCalculator`, so behaviour is swappable by config alone.
- **Measure the blind-vs-sighted gap before trusting any training number** on a new Ground. This is M3's hardest-won lesson and it generalises.
- If a future Ground needs finer spatial detail, the documented next rung is **unfreezing the last ViT block** — not editing PPO.